

CM2005 Object Oriented Programming Mid Term Report

By Ngon Nguyen Below is my report for the mid term project of CM2005, written on July 11 2023. This report will detailed out how I carried out each tasks. Please also note that inside each source code file and header files, all parts that are personally wrote by me without assistance are preceded by the comment `// ADDITION #num` with `num` being the numbering I assigned to keep track of what I wrote myself, not necessarily their order of creation.

WARNING: This project utilises ANSI color codes and Unicode glyphs in the terminal output for aesthetic purposes, and not every terminals support such features. The project has been thoroughly tested on my Ubuntu 22.04 laptop with the Kitty terminal running bash, and was proven to work well under such circumstances, as evidenced by the video submitted (July 14 Update: Also tested to be working well with VS Code on Windows 10 using git-bash terminal, after enabling the option mentioned below). I do not, however, guarantee it might work in other OS or terminals. For Windows, please refer to [this StackOverflow thread](#) on how to enable the Beta: Use Unicode UTF-8 for worldwide language support option on Windows 10 to see those Unicode glyphs displayed correctly.

NOTE: All hyperlinks in the text below, when hovered upon, will show the relative path to the referenced file. For example, [this file](#). However, clicking on them might not take you to the exact file, due to some limitations in the conversion of this Markdown file to PDF by pandoc. Please open the file manually if need be.

TASK 1: Compute Candlestick Data

This was the most straightforward and easiest task to complete. Based on the definition in the assignment PDF, all info required to compute Close, High and Low of a certain candlestick of a certain timeframe is already inside that timeframe itself. Only the Open is a bit tricky, as it requires data from the previous timeframe.

It was here that another problem arose, what is considered to be a “timeframe”? This definition varies from one person to the next, and real-life crypto exchanges such as Binance or Coinbase allows you to choose your own. On Binance, for example, you can choose 15 minutes, an hour or a day as a timeframe for your candlestick, among others. However when I take a look at the source datasets, I quickly noticed that the whole dataset only lasted for minutes at most, meaning if I choose a timeframe as big as a minute, there won’t be many such candlesticks to draw. However, I also noticed that the orders are snapshotted at 5-second intervals. Even the `MerkelMain` itself is advancing the app by 5s everytime the user chose the option to continue. Therefore, I’ve made the decision to choose

my timeframe as a block of 5 seconds, as it will fit in nicely with the rest of the software.

Now it was time to solve the matter of Open. It needed a previous timestamp to calculate its own value for this timestamp. Meaning, the very first timestamp that we started out with will not have such a preceding timestamp, and it would be impossible to calculate its Open value. In the course videos, professor Matthew Yee-King did at least once doing the loop-over (going back to the first line of the dataset when one loop hit the last line). I considered doing that, but then decided against it, as it is both illogical (time flow forward, not backwards) and impractical (the final order's stats may be drastically different from the first and may skew its value in an unrealistic way e.g. massively inflate the opening price even beyond the High price, etc.). Therefore, I made the choice to just return 0 for the value of Open for the very first entry, and print out a message to warn users of such missing data, encouraging them to proceed the time flow to next timeframes.

With that solved, the rest was quite straightforward. I realized because the calculation steps for Open and Close values are exactly the same (the only difference being the timestamp used in the calculation), there was no need to write 2 separate methods for it, so I make a single method called `getOpeningAndClosingPrice()`, and pass in a vector of `OrderBookEntry` that was harvested based on the timestamp. This vector, in turn, was collected by using the `getOrders()` method created by professor Yee-King previously, filtered on the `OrderBookType`, product names, and timestamps, etc.

The final calculation method was named `computeCandlesticks()`. It returns a vector of Candlesticks data to be drawn as a graph and requires input: a pair of product names, an `OrderBookType` type (`bid/ask`) and the current timestamp. Its inner workings were explained in great detail in the source code file `Orderbook.cpp`, from line 163 to line 219, so please reference that as I wish not to further cluttering it up in this report by repeating it again here.

TASK 2: Create a text-based plot of the candlestick data

This was a huge assignment and a half. From the very beginning, I was surrounded by 2 main problems waiting to be solved:

1. I don't even have the slightest idea of what it should look like
2. I have the candlestick data (Open, High, Low, Close), but don't know how to represent them as shapes

I try to tackle the 1st 2 problems by doodling a bit, with the `result` encased in the source code zip file (also with some crude problems I needed to solve at that stage, please refer to the file for further info). After finding a satisfactory set of visuals to aim for (huge thanks to w3.org for listing out some of the best and most various Unicode character sets ever), I started to draw a rudimentary graph, starting with the Cartesian X and Y axes.

My first instinct was to use `std::cout` to output characters. I know that `cout` will not output everything in a “smart” way like JavaScript’s `console.log()` or Python’s `print()` i.e. everything on a new line, instead it will only print out whatever I tell it to. So, my reasoning was: “If I go into a nested for loop with `i` and `j`, and on `j = 3`, `i = 0` I output a `|` character, then at `i = 0`, `j = 4` I output another such character, I would have a vertical line spanning 2 rows, right?”

Well, yes. And no. That line of reasoning worked great for outputting simple shapes such as lines, and I drew up the X and Y axis in no time. However, as the shapes get more and more complicated (markers on X and Y axes, text and number entries next to those markers, ...) the for loop got more and more bloated (nearly 200 lines at one point) and difficult to manage. Not only do you have to keep in mind which `cout` in which part of the for loop was responsible for what, the order in which they come in was also a huge headache. Things that go later in the loop would overwrite things that came before them. But I kept plugging at it.

Until I tried to draw the candlesticks themselves. And everything fell apart like a house of cards. You see, `cout` can only **append** things to the standard output, it can not **replace** them. And as a result, “blank space management” reared its ugly head as the next impossible obstacle to overcome. Only difference is this time, no matter what I tried, I couldn’t overcome it. Now looking back at it, I know why. I was trying to change the nature of `cout` itself, I was trying to make it replace certain symbols with others (i.e. making the candlestick overwrite part of the stalk) while `std::cout` could only append to whatever it has already outputted. But back then ... no matter what I did, something would look off, things would get pushed one, two, three spaces to the right. That was because my nested for loop looked something like this pseudocode:

```
//silly, I know, should have used x and y
//from the beginning for clarify sake
for (j = 0; j < HEIGHT; j++){
    for (i = 0; i < WIDTH; i++){
        if
            //drawing code
        else if
            //drawing code
        else
            std::cout << " "
    } std::endl;
}
```

Basically, I had to make sure that problematic `else` statement output a space at the exact location without fail, just a single extra blank space and it would push everything else to its right one spot to the right, messing up the whole graph’s look. That’s what I called “blank space management” and it took me nearly a week to eventually come to the realization that even though I can maybe draw a passable candlestick graph using this method, it would take an ungodly amount

of hours and effort to do so, and would certainly **NOT** be even remotely modular enough to be packaged into OOP classes.

That's how I ended up spending a whole day to rethink my approach and came up with the `drawCanvas` vector. This is a 1-dimension array that would store `Coord`(ination) objects. A `Coord` object is a simple class consisted of one int x and one int y. My game plan was to keep the nested loop (finally with iterators renamed into x and y for clarity's sake), and whenever it found something in `drawCanvas` that has matched x and y, a `std::cout` would be called to output that character to standard output. Something like this:

```
for (y = 0; y < HEIGHT; y++){
    for (x = 0; x < WIDTH; x++){
        for (z = 0; z < drawCanvas.size(); z++){
            if (x == drawCanvas[z].getX() && y == drawCanvas[z].getY()){
                std::cout << drawCanvas[z].getSymbol();
                //back then, the Coord class also has a "symbol"
                //private member that stored what the coordinate
                //should write out to cout
            }
            else {
                std::cout << " ";
            }
        }
    }
}
```

Again, this worked well for a while, until I realized the “blank space management” problem still existed. Even though the new drawing code is amazingly easy to read, every single space in it still controlled by that unreliable `else -> cout << " "` statement. It also needed a separate nested loop above it to create and push every `Coord` object to the `drawCanvas`, not great for performance. Oh, what ...? The draw loop itself is a 3-level deep nested loop? That's $O(n^3)$ then, horrible, horrible for performance. To add insult to injury, I realized that this new code that I spent a whole day recreating, is exactly the same as the one before it, except for the fact that all the `std::cout` drawing code is grouped at one place.

It was at that exact dark moment that I looked at the name `drawCanvas` and had an Eureka moment. I thought about `P5.js` and its canvas how the canvas was simply just a 2D array of white pixels, and when you want to draw something, you set each pixel to a certain color or shape. Simple at that. Could I repeat that in C++? 2D array? - check, it's called “vectors” over here, pixels? check, just make the vector store strings (I actually chose char at first, but some characters just won't fit in that data structure), white (blank) pixels? check, just initialize everything in the 2D vector to be a blank space.

And that was how the `Canvas` class was born. Now that I had a `Canvas` class,

everything else was so easy. I kept the logic of the previous drawing loops, but implemented them based on the `Canvas` class and shapes just appeared one after another so very effortlessly, beautifully and perfectly (aligned). I just need to set a pixel to a symbol and it stayed that way, because everything else around it was already a blank space. No need to rely on unreliable `else` statement that do `std::cout << " "!`

Now that I got the drawing part down, it was time to move on to *what* to draw. I realized everything would be so simple if I only have one candlestick, just put its High as the max point for Y axis, Low at the min point, and align its Close and Opening to respective points on the Y axis. However, when many candlestick have to be drawn in the same Cartesian plane, things got tricky. First, I have to decide what number the min and the max point on Y axis point should represent, which was straightforward enough, just traverse through the entries `Candlestick` vector and find a max and min value. But then, how should I represent each individual candlestick based on those min and max values?

I spent a whole day thinking about that and found another solution, also from P5.js: the `map()` function, which takes in a value, an input range, and output range, and output that same value, mapped to the output range. By implementing my own [mapping](#) function, I was able to map the candlestick values from their original form into a number that belonged in my Cartesian system. From then on, it was just a matter of writing the necessary code to help move everything else along, then when I got the visualisation I wanted from a monolithic file, I started chopping it up into smaller methods, then logically distributing the methods to existing classes such as `getHigh()` and `getLow()` into the `Candlestick` class as they accept a vector of `Candlesticks` and extract certain values from them, or `convertCandlestickToCoord()` to `Coord` class.

The rest of what I couldn't logically fit into anything else and logically belong together, such as the drawing (setting pixels) code, graph-specific constants, etc. I bundled them up into a new class called `CandlestickGraph`. As the logical extension of a `Canvas` (a graph is drawn on a blank canvas, right?) this class is a child of the `Canvas` class and inherit all of the latter's methods and protected members (I tried setting them to private at first, but then they won't allow me to access them from the children class. Therefore, protected it is, lesson learned). This `CandlestickGraph` class also included things that doesn't logically fit in a `Canvas` but does in a graph (such as `setXAxis()` or `completeYAxis()`).

Finally, the very last of the stragglers that doesn't fit in absolutely anywhere else logically, I put them in a file called `Helpers.cpp`. My first instinct was to make this a class as well. However, I decided against it as those helper functions are used quite widely throughout the project and creating a `Helper` object everytime I want something quick and simple isn't an efficient use of memory. I also don't want to create a class full of static objects only, it just feels wrong to create an empty class with only static objects, also feels overly complicated while I can achieve the same effect with just function prototypes.

After the drawing was done, I was confronted with another problem:

3. How do I precisely align the candlesticks on the graph to their actual value?

You see, my first iteration of the `CandlestickGraph` was very crude in this regard. In fact, it was so crude that I didn't even round the mapped numbers properly. For example, if a Y value of a point on a candlestick was mapped to be 26.6, I would have just casted it directly to an integer without a second thought, only taking the integer part of the double, which was 26, and a long way from the actual value. I then realized this negligence and tried to right it by including the `<cmath>` library and perused its `round()` function to round it to the closest integer possible, which was a bit better, but still not as precise as I would have liked. My graphs looked great, but then if you look closely, it was wildly inaccurate, and that certainly doesn't sit right with me.

So I dig a bit deeper, and found out that in the Unicode character set I used, a full block character does have variants that are fractions of it. Better yet, they even have variants that are exactly the same, only upside down. Armed with this new knowledge, I set out to make the precision to be at least one eighth of a block. In order to do this, I first change every `double` variable in the project into `long double` to increase the precision (especially with DOGE, their value is just so small against everything else), then round the mapped value up or down depended on their orientation (at the top or bottom of the candlestick body), then take out the remaining (using functions such as `processMappingUpper()` and `processMappingLower()`), and compare the remaining part to the fractions mentioned above (using `processRemainder()`), and represent it on the `CandlestickGraph` as the closest fraction possible. This resulted in a slightly worse looking graph (as the fractions doesn't connect to the stalk seamlessly, they are, Unicode characters after all, and 2 characters can't share the same letter space), but I feel it's a necessary sacrifice for a vastly more precise graph.

TASK 3: Plot a text graph of some other trading data

With all cornerstones such as `Canvas` and `CandlestickGraph` already in place, I now have the creative freedom to look for some other trading data to plot, and I've decided upon the trading volume of each product. This style of graph is widely represented in 3 ways: as a pie chart, as a line plot or a bar graph. Out of the 3, I chose to represent it as a bar graph as it's the most straightforward to be represented within my existing infrastructure.

The first thing I need to do is basically recreate what I did with the `Candlestick` graph: creating a basic class of objects to store the most basic unit on the graph, similar to the `Candlestick` class. That was the `Volume` class, which only consisted of a single product name and its trading volume. This trading volume was computed using the method `computeVolumes()`, that worked in a pretty similar way to `computeCandlesticks()` mentioned in TASK 1, albeit due to

the simpler class Volume compared to Candlestick (much less moving parts), `computeVolumes()` is considerably less complex than `computeCandlesticks()`.

It's also worth mentioning that I meant to convert everything to USDT (using the stablecoin as the unit of measuring trading volume), therefore I also considered trading pairs such as ETH/BTC as trading volume for **both** ETH **and** BTC (in pairs with USDT such as ETH/USDT it's straightforward because we only need to count it as ETH volume, as USDT volume is always equal to 100% of the trading volume anyway).

Afterwards, I just need to create a copy of the CandlestickGraph class, rename it to VolumeGraph (meaning it's also a child to the Canvas class, which is logical), clear out a significant portion that doesn't apply to the VolumeGraph such as `candleCoords` or `ranges` and I'm left with a solid class able to draw what I want in an effortless manner.